Customer Churn Management

A Relational Database Solution

Valencia Vincent Dmello
Department of Computer Science and Engineering
University at Buffalo
New York,USA
vdmello@buffalo.edu

Tarun Ambaldhage
Department of Computer Science and Engineering
University at Buffalo
New York,USA
tarunamb@buffalo.edu

***Abstract*—This project aims to address customer churn management for a streaming service by designing a relational database system to manage subscriptions, billing, and content preferences. The implementation replaces traditional Excel spreadsheets to enhance data integrity, scalability, and query efficiency. The database schema includes three interconnected tables—Customer, Subscription, and Usage—to capture comprehensive user data. The project also includes a Streamlit application for querying and analyzing customer behavior. This report outlines the schema design, SQL implementation, and insights derived from the system.**

***Index Terms*—Relational Database Design, SQL Implementation, Data Integrity, Streaming Services, Subscription Analytics, Usage Patterns, Streamlit Application, Customer Retention, Query Optimization.**

I. Introduction

Customer churn, defined as the rate at which customers stop using a service, poses significant challenges for streaming platforms. This project addresses these challenges by creating a relational database to track customer behavior, billing, and preferences. By analyzing the data, the system helps identify patterns leading to churn, enabling retention strategies.

A Relational database is better suited for this system for following benefits:

- Data Integrity: We can ensure data consistency across all tables, reducing the chances of duplicate entries and other discrepancies.
- Scalability: It can handle growing numbers of users and increasing amounts of data without sacrificing performance. It can also perform transactions as the service grows without slowing down.
- Query Efficiency: It allows us to run complex searches to get insights quickly, which would be difficult and time-consuming with Excel.
- Data Security: It offers better ways to protect sensitive customer information, such as billing information and personal details, compared to Excel files.

II. Database Design

A. Schema Overview

The database schema comprises the following tables:

- Customer: Stores demographic and account information.
- Subscription: Contains subscription plans, billing, and churn status.
- Usage: Tracks interaction metrics like viewing hours and content preferences.

Key Features:

- Relational Design: Tables are connected via foreign keys (CustomerID) to maintain data integrity and relationships between entities. One-to-Many relationships are implemented. A customer can have multiple Usage records. A customer can have multiple Subscription records.
- Normalization: Data is normalized to avoid redundancy. For example, subscription and usage details are in separate tables, linked to the Customer table.
- Expandability: The structure allows for adding more attributes or tables (e.g., billing history, customer support interactions) without redesigning the entire schema.

B. Table Descriptions

- Customer Table:
  - CustomerID: Unique identifier for each customer (integer)
  - SubscriptionType: Type of subscription the customer has (varchar)
  - PaymentMethod: Method the customer uses to pay(varchar)
  - PaperlessBilling: Whether the customer has opted for paperless billing(varchar)
  - Gender: Customer's gender(varchar)
  - AccountAge: Number of months the customer's account has been active(integer)
  - UserRating: Rating provided by the user for the streaming service(integer)

* WatchlistSize: Number of items in the customer's watchlist(integer)

– Subscription Table:

* SubscriptionTypeID: Unique identifier for the subscription(integer)
* CustomerID: Foreign key referencing CustomerID in the Customer table(integer)
* UsageID: Foreign key referencing UsageID in the Usage table(integer)
* MonthlyCharges: Monthly charges for the customer(integer)
* TotalCharges: Total charges accumulated by the customer(integer)

– Usage Table:

* UsageID: Unique identifier for the usage data(integer)
* CustomerID: Foreign key referencing CustomerID in the Customer table(integer)
* ContentType: Type of content the customer consumes(varchar)
* MultiDeviceAccess: Whether the customer has access to multiple devices(varchar)
* DeviceRegistered: Type of devices registered by the customer(varchar)
* GenrePreference: Preferred genre of the customer(varchar)
* ViewingHoursPerWeek: Number of hours the customer views content per week(integer)
* SupportTicketsPerMonth: Number of support tickets raised by the customer per month(integer)
* AverageViewingDuration:Average duration the customer spends watching content (integer)
* ContentDownloadsPerMonth: Number of content items downloaded by the customer per month(integer)

C. Entity-Relationship (E/R) Diagram

The E/R diagram depicts relationships between the tables, emphasizing the hierarchical structure for seamless integration of customer, subscription, and usage data.

Customer ↔ Usage:
- Relationship: A customer can have multiple usage records.
- Key: CustomerID in the Usage table links to CustomerID in the Customer table.
- Cardinality: 1-to-Many (One customer can have multiple usage records).

Customer ↔ Subscription:
- Relationship: A customer can have multiple subscriptions.
- Key: CustomerID in the Subscription table links to
- CustomerID in the Customer table.
- Cardinality: 1-to-Many (One customer can have multiple subscriptions).

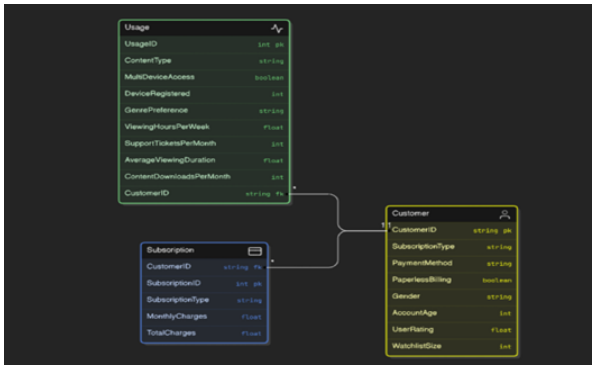

Fig. 1. ER Diagram

D. Default values and Nullable Attributes

- PaperlessBilling: Default set to 'No' as this is often an optional choice.
- AccountAge: Default is set to 0 as it's mandatory, starting from account creation.
- Fields where values are not required and can be left empty (e.g., SubscriptionType, PaperlessBilling, ContentType).

III. Implementation

A. SQL Scripts

1) create.sql:

This SQL script defines a database schema for a customer churn management system. CREATE DATABASE creates a new database named customer_churn. USE customer_churn switches to the newly created database, making it the active database for subsequent operations.

2) load.sql:

This SQL script populates the Customer, Usage, and Subscription tables by extracting and transforming data from a source table called train

B. Streamlit Application

- Query Execution: The app.py file creates a Streamlit application that allows users to view database tables, execute custom SQL queries, and visualize query results in real-time. It connects to an SQLite database, fetches data from tables, and provides an

interface for running user-defined SQL queries.The results are then processed and displayed in a user-friendly format, such as tables or graphs. This feature allows users to explore specific data subsets based on their needs (e.g., viewing hours, churn rates, or customer preferences).

- Creating a Git Repository: To ensure version control and facilitate collaboration, the entire project, including the app.py file and all related assets, was stored in a Git repository. This allowed the team to track changes, handle code revisions, and maintain an organized project history.
- Deploying the Application: The Streamlit application was deployed using GitHub and connected to a hosting platform Streamlit Cloud. The deployment process involved pushing the app.py file to the GitHub repository, ensuring that the application could be run remotely. Once deployed, users could access the application via a URL provided by the hosting platform.
  Click here to visit https://customerchurn-jk7zemhxmuvpycbuxwxpic.streamlit.app/
- Running the Application: Once the application was deployed, users could access it through a web interface and interact with the data in real-time. The application was designed to be user-friendly, allowing for seamless navigation and exploration of the database.

IV. Results and Analysis

A. Customers with fewer than five viewing hours per week

```
SELECT
AVG(CASE WHEN u.ViewingHoursPerWeek < 5
THEN 1 ELSE 0 END) AS ChurnRate_LowViewing
FROM
    Customer c
JOIN
    Usage u
ON
    c.CustomerID = u.CustomerID;
```

	ChurnRate_LowViewing
0	0.1018

The churn rate for customers with fewer than five viewing hours per week is 10.18This is relatively low, indicating that while low usage could suggest potential churn, the impact isn't as high as expected (30% higher). It could be because other factors (such as satisfaction, support, or subscription type) play a significant role in churn.

B. Customers using multiple devices

```
SELECT
AVG(CASE WHEN u.MultiDeviceAccess = 'Yes'
THEN 1 ELSE 0 END) AS ChurnRate_MultiDevice
FROM
    Customer c
JOIN
    Usage u
ON
    c.CustomerID = u.CustomerID;
```

	ChurnRate_MultiDevice
0	0.4994

The churn rate for customers using multiple devices is 49.94%, which is notably high. This contradicts the assumption that customers using multiple devices would churn less. It suggests that, rather than reducing churn, these customers may be more engaged in exploring the service across various platforms, leading to higher churn due to issues like dissatisfaction, cost, or other factors that could increase usage but decrease loyalty.

C. Premium subscribers rate compared to basic plans

```
SELECT
    c.SubscriptionType,
AVG(CASE WHEN u.ViewingHoursPerWeek < 5
THEN 1 ELSE 0 END) AS ChurnRate,
SUM(s.MonthlyCharges) AS TotalRevenue
FROM
    Customer c
JOIN
Subscription s ON
    c.CustomerID = s.CustomerID
JOIN
Usage u ON c.CustomerID = u.CustomerID
GROUP BY
    c.SubscriptionType;
```

	SubscriptionType	ChurnRate	TotalRevenue
0	Basic	0.1019	1,012,894.18
1	Premium	0.1035	1,008,730.10
2	Standard	0.0999	1,023,444.66

- Basic Plan: Has a relatively low churn rate of 10.19%, and it contributes to a significant portion of the total revenue (1,012,894.18). This indicates that even though customers on the Basic plan have a similar churn rate to Premium, it generates substantial revenue.
- Premium Plan: Has a slightly higher churn rate (10.35%) compared to Basic but a similar level of revenue (1,008,730.10). Premium plans are typically expected to have better retention, but the higher churn rate might suggest that customers are leaving for better value elsewhere, despite paying more.
- Standard Plan: Has the lowest churn rate (9.99%) and generates the highest total revenue (1,023,444.66). This suggests that the Standard plan strikes a good balance between customer retention and revenue generation.

D. Genre preferences revealed a strong correlation

```sql
SELECT
    u.GenrePreference,
    AVG(u.ViewingHoursPerWeek)
    AS AvgViewingHoursPerWeek
FROM
    Usage u
GROUP BY
    u.GenrePreference
ORDER BY
    AvgViewingHoursPerWeek DESC;
```

	GenrePreference	AvgViewingHoursPerWeek
0	SciFi	20.5543
1	Action	20.5372
2	Fantasy	20.5022
3	Drama	20.474
4	Comedy	20.444

* The genres SciFi, Action, and Fantasy have very similar and high viewing hours, all over 20 hours per week.
* Drama and Comedy also have significant engagement, though slightly less than SciFi and Action.
* These results suggest that SciFi and Action genres are particularly popular among viewers, and these genres might have a strong correlation with customer retention, as high engagement typically translates to greater satisfaction and less churn.

V. Conclusion

This project developed a relational database and application to manage customer churn in a streaming service. The system ensures data integrity, supports efficient queries, and generates actionable insights.
Future Enhancements:
Integrate machine learning models for predictive analytics (e.g., churn likelihood). Expand the schema to include customer support logs and social media interactions. Enhance the application to offer graphical insights such as churn trends and revenue forecasts.

VI. References

1) Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*, 2020, McGraw-Hill.
2) Streamlit Documentation, "Building Custom Data Applications," 2024, https://docs.streamlit.io/ (Accessed: 2024-12-02).
3) MySQL Documentation, "SQL Constraints and Relationships," 2024, https://dev.mysql.com/doc/refman/8.0/en/ (Accessed: 2024-12-02).